

Pokémon ETL Pipeline — Junior Data Engineer Project

A self-contained ETL project that teaches the three core skills of data engineering: **ingestion**, **transformation**, and **loading**. You will pull data from the public PokéAPI, shape it with Polars, and land it in an analytical store you can query with SQL.

By the end of this project you should be comfortable explaining — and defending — every choice you made in your pipeline.

1. Learning objectives

Area	What you will learn
Ingestion	HTTP clients, pagination, retries, exponential backoff, rate-limit etiquette, caching, raw data preservation
Transformation	Polars <code>DataFrame</code> / <code>LazyFrame</code> API, schema design, joins, aggregations, struct/list handling, nested JSON flattening
Loading	Parquet, partitioning, DuckDB analytical store, idempotent writes, the <i>raw</i> → <i>clean</i> → <i>mart</i> (medallion) pattern
Engineering	Project layout, configuration, logging, typing, basic testing, makefiles / task runners

2. Tech stack

- **Python 3.11+**
- **httpx** — HTTP client (sync to start, async as a stretch goal)
- **tenacity** — retries with backoff
- **polars** — transformation engine
- **duckdb** — analytical SQL database (no server, just a file)
- **pydantic** (*optional*) — validation of API responses
- **pytest** — tests
- **ruff + mypy** (*optional*) — linting and typing

┃ No paid services. No Docker required. No cloud account needed.

3. The data source — PokéAPI

PokéAPI is free, unauthenticated, and well-documented. Base URL:

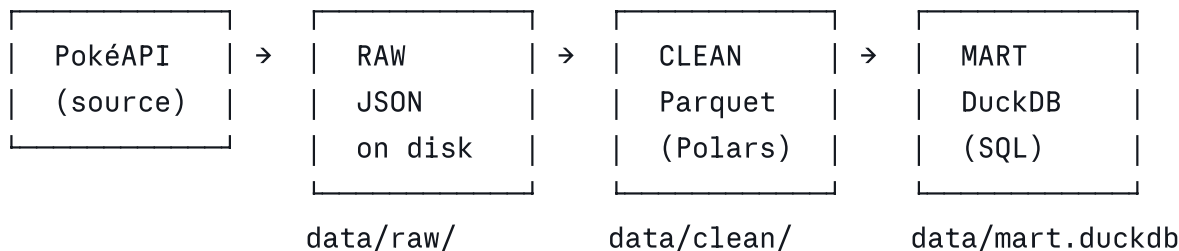
`https://pokeapi.co/api/v2/`.

You will work with these resources:

Endpoint	What it gives you
<code>/pokemon?limit=...&offset=...</code>	Paginated list of Pokémon (name + URL)
<code>/pokemon/{id}</code>	Stats, types, abilities, height, weight, base experience
<code>/pokemon-species/{id}</code>	Generation, habitat, capture rate, flavour text
<code>/type/{id}</code>	Damage relations between types
<code>/ability/{id}</code>	Ability descriptions
<code>/generation/{id}</code>	Pokémon released in each generation

Be a good citizen: the API has no hard rate limit but explicitly asks you to **cache responses locally** and avoid hammering it. Your ingestion layer must respect that.

4. Architecture — the medallion pattern



- **Raw:** byte-for-byte copy of every API response. Never overwrite. Re-runnable without re-fetching.
- **Clean:** typed, flattened, deduplicated Polars-friendly Parquet files. One file per entity.
- **Mart:** analyst-ready tables in DuckDB with foreign-key-style joins resolved.

5. Project structure

```
pokemon-etl/  
├─ README.md  
├─ pyproject.toml          # or requirements.txt  
├─ Makefile               # convenience targets: make extract, make transform,  
...  
└─ ...
```

```

└─ config/
  └─ settings.py          # base URL, paths, batch size, etc.
└─ src/
  └─ pokemon_etl/
    └─ __init__.py
    └─ extract/
      └─ client.py        # HTTP client with retries + cache
      └─ ingest.py        # orchestrates which endpoints to pull
    └─ transform/
      └─ pokemon.py       # one module per entity
      └─ types.py
      └─ species.py
    └─ load/
      └─ duckdb_loader.py
    └─ pipeline.py        # ties extract → transform → load together
    └─ cli.py             # `python -m pokemon_etl`
extract|transform|load|all`
└─ data/
  └─ raw/                 # gitignored
  └─ clean/               # gitignored
  └─ mart.duckdb          # gitignored
└─ tests/
  └─ test_client.py
  └─ test_transform_pokemon.py
  └─ fixtures/
    └─ pokemon_25.json

```

6. Milestones

Milestone 1 — Extract (ingestion)

Build `extract/client.py` and `extract/ingest.py` so that:

1. A single function `get(path: str) -> dict` performs a GET against PokéAPI.
2. It **retries** transient failures (`5xx`, connection errors) with exponential backoff — use `tenacity`.
3. It **caches** every response to `data/raw/<resource>/<id>.json`. If the file already exists, return it from disk instead of hitting the network. This is your idempotency guarantee.
4. It **paginates** through `/pokemon` using `limit` / `offset` until exhausted.
5. It logs (not prints) every request with status code and source (cache vs network).

Deliverable: running `python -m pokemon_etl extract` populates `data/raw/` with JSON for every Pokémon, species, type, and ability. A second run does no network calls.

Stretch: rewrite the client with `httpx.AsyncClient` and an `asyncio.Semaphore` to fetch ~20 in parallel. Compare wall-clock time.

Milestone 2 — Transform (Polars)

For each entity, write a function that reads the raw JSON and returns a Polars `DataFrame` with an explicit schema. Save each as Parquet in `data/clean/`.

Required output tables:

`pokemon.parquet`

column	type	source
pokemon_id	Int32	<code>id</code>
name	Utf8	<code>name</code>
height_dm	Int32	<code>height</code>
weight_hg	Int32	<code>weight</code>
base_experience	Int32	<code>base_experience</code>
is_default	Boolean	<code>is_default</code>

`pokemon_stats.parquet` — one row per (pokemon, stat). This is your first taste of **unnesting list-of-structs**, the bread and butter of Polars.

`pokemon_types.parquet` — one row per (pokemon, slot, type). Same pattern.

`pokemon_abilities.parquet` — one row per (pokemon, ability, is_hidden).

`types.parquet` + `type_damage_relations.parquet` — model the damage-multiplier graph between types (e.g. water → fire = 2x).

`species.parquet` — pokemon_id, generation_id, capture_rate, is_legendary, is_mythical.

Hard requirements:

- Declare schemas explicitly with `pl.Schema(...)`. Do not let Polars infer types.
- Use `LazyFrame` and `.collect()` only once at the end of each pipeline.
- Use `pl.col(...).struct.field(...)` and `.list.explode()` rather than Python loops over rows.
- Add at least one **derived column**: e.g. `bmi = weight_kg / (height_m ** 2)`, or a `stat_total` aggregation.

Stretch: validate raw JSON with Pydantic models before transforming.

Milestone 3 — Load

Build `load/duckdb_loader.py` so that `python -m pokemon_etl load`:

1. Opens `data/mart.duckdb`.
2. For each Parquet file in `data/clean/`, registers it as a DuckDB table — `CREATE OR REPLACE TABLE pokemon AS SELECT * FROM read_parquet('data/clean/pokemon.parquet')`.
3. Creates at least **three analyst-ready views** that pre-join the data:
 - `vw_pokemon_full` — pokemon + species + a comma-joined list of types
 - `vw_type_matchups` — readable type-vs-type damage multipliers
 - `vw_stats_pivot` — one row per pokemon, one column per stat (hp, attack, defense, ...)

Acceptance test: the following queries must run and return sensible results:

```
sql

-- Heaviest Pokémon per type
SELECT type_name, pokemon_name, weight_hg
FROM vw_pokemon_full pf JOIN pokemon_types pt USING (pokemon_id)
QUALIFY ROW_NUMBER() OVER (PARTITION BY type_name ORDER BY weight_hg DESC) = 1;

-- Average stat total by generation
SELECT generation_id, AVG(stat_total) AS avg_total
FROM vw_stats_pivot JOIN species USING (pokemon_id)
GROUP BY generation_id ORDER BY generation_id;

-- Which type is strong against the most other types?
SELECT attacking_type, COUNT(*) AS num_super_effective
FROM vw_type_matchups WHERE multiplier = 2.0
GROUP BY attacking_type ORDER BY 2 DESC;
```

Milestone 4 — Orchestrate & operationalise

- Wire everything into `pipeline.py` so `python -m pokemon_etl all` runs extract → transform → load in order.
- Add a `Makefile` (or `justfile`) with `make extract`, `make transform`, `make load`, `make all`, `make clean`, `make test`.
- Add **structured logging** (`logging` module, JSON formatter optional) with `INFO` showing milestones and `DEBUG` showing per-record detail.

- **Add at least three tests:**

1. `test_client.py` — mock the HTTP layer, assert the cache works on the second call.
 2. `test_transform_pokemon.py` — given a fixture JSON, assert the resulting DataFrame has the expected schema and row count.
 3. `test_pipeline.py` — end-to-end on a tiny subset (e.g. only Pokémon 1-10), asserting the final DuckDB tables exist and are non-empty.
-

7. Stretch goals (pick any)

- Swap DuckDB for Postgres via `duckdb`'s `ATTACH` or SQLAlchemy. Compare load times.
 - Incremental loads — track which IDs have been ingested in a state table and only fetch new ones.
 - Run the whole pipeline in GitHub Actions on a schedule.
 - Add data quality checks with Great Expectations or hand-rolled Polars assertions (e.g. *no nulls in `pokemon_id`, every pokemon has at least one type*).
 - Build a small Streamlit dashboard on top of the DuckDB file.
 - Containerise with Docker; mount `data/` as a volume.
-

8. Setup

```
bash

git clone <your-fork>
cd pokemon-etl
python -m venv .venv && source .venv/bin/activate
pip install -e ".[dev]"      # or: pip install -r requirements.txt

# Run the whole thing
python -m pokemon_etl all

# Or step by step
python -m pokemon_etl extract
python -m pokemon_etl transform
python -m pokemon_etl load

# Query the result
duckdb data/mart.duckdb -c "SELECT COUNT(*) FROM pokemon;"
```

Suggested `requirements.txt`:

```
httpx>=0.27
tenacity>=8.2
polars>=1.0
duckdb>=1.0
pydantic>=2.0
pytest>=8.0
```

9. Evaluation criteria

You will be reviewed on the following — not just on whether it works:

Dimension	What "good" looks like
Correctness	All milestone deliverables run end-to-end; tests pass; analyst queries return sensible numbers.
Idempotency	Running the pipeline twice produces the same final state. No duplicates. No unnecessary network calls.
Code quality	Clear module boundaries, small functions, type hints, no Polars in the extract layer, no HTTP in the transform layer.
Polars idiom	Lazy evaluation, explicit schemas, vectorised operations (no <code>for</code> loops over rows, no <code>.iter_rows()</code>).
Resilience	Network failure mid-run does not corrupt <code>data/raw/</code> and can be resumed by re-running.
Documentation	Your own <code>README.md</code> explains the architecture, how to run it, and one trade-off you made.

10. Hints & gotchas

- The PokéAPI returns **1302+ Pokémon** as of 2025 (including variants). A full extract takes a few minutes on first run, near-zero on cached runs.
- `pokemon.stats` is a list of structs — `pl.col("stats").list.eval(pl.element().struct.field("base_stat"))` is your friend.
- `height` is in decimetres and `weight` in hectograms — divide by 10 for metres and kilograms. Document this in your column names or comments.
- DuckDB can read Parquet directly: `SELECT * FROM 'data/clean/pokemon.parquet'`. Use that during development to avoid re-loading.

- Don't commit `data/` to git. Add it to `.gitignore`.
 - If you find yourself writing `requests.get` inside a transform function, stop and re-read the architecture section.
-

11. Timebox

Roughly two weeks of part-time work for a junior engineer new to Polars. Suggested split:

- **Days 1-3:** project scaffold + Milestone 1 (extract)
- **Days 4-8:** Milestone 2 (transform) — this is where most of the learning happens
- **Days 9-11:** Milestone 3 (load) + analyst queries
- **Days 12-14:** Milestone 4 (tests, logging, polish) + one stretch goal of your choice

Good luck — and try to get the pipeline working end-to-end on Pokémon 1-10 *before* you scale it to the full dataset. Fast feedback loops will save you days.